

Звіт

до Лабораторної роботи №16

Автоматизована збірка, тестування і CI для Java-проєкту

Студент: Колісніченко Артем Олександрович

Група: КН-924в

Викладач: Савченко В. М.

Рік: 2026

1. Тема і мета роботи

Тема лабораторної роботи — автоматизована збірка, тестування і CI для Java-проєкту.

Метою роботи є набуття практичних навичок підготовки Java-проєкту до автоматизованої збірки, запуску тестів і безперервної інтеграції, а також засвоєння базових принципів інтеграції Java-коду з native-кодом за допомогою JNI.

У роботі Java-проєкт розглядається як інженерний артефакт, який повинен коректно збиратися, тестуватися локально та перевірятися у віддаленому середовищі GitHub Actions.

2. Постановка індивідуального завдання

Варіант 5 — нормалізація значень

За індивідуальним варіантом потрібно реалізувати операцію нормалізації масиву чисел до діапазону 0..1.

Операція повинна бути реалізована у двох версіях:

- звичайною мовою Java;
- мовою C++ через JNI.

Обидві реалізації повинні повертати однаковий результат для однакових вхідних даних.

Формула нормалізації:

$$(x - \min) / (\max - \min)$$

Якщо всі значення масиву однакові, результатом є масив нулів. Це дозволяє уникнути ділення на нуль.

3. Опис реалізації

У проєкті створено класи `JavaNormalizer`, `NativeNormalizer`, `NormalizerService` і `Main`.

Клас `JavaNormalizer` містить звичайну Java-реалізацію операції нормалізації. Спочатку в масиві знаходяться мінімальне та максимальне значення, після чого кожен елемент перетворюється за формулою нормалізації.

Клас `NativeNormalizer` оголошує `native`-метод і завантажує `native`-бібліотеку `native-normalizer`. Реалізація цього методу знаходиться у `C++` файлі.

Клас `NormalizerService` об'єднує Java- та `native`-реалізації. Він дозволяє викликати обидва варіанти нормалізації та сформувати текстовий звіт для демонстрації результатів.

Клас `Main` використовується для демонстраційного запуску програми.

4. Структура проєкту

Проєкт містить Java-код, `native`-код, тести, UML-діаграми, Maven-конфігурацію та workflow GitHub Actions.

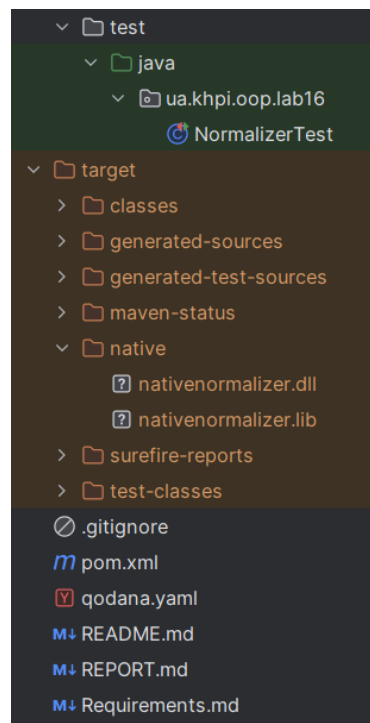
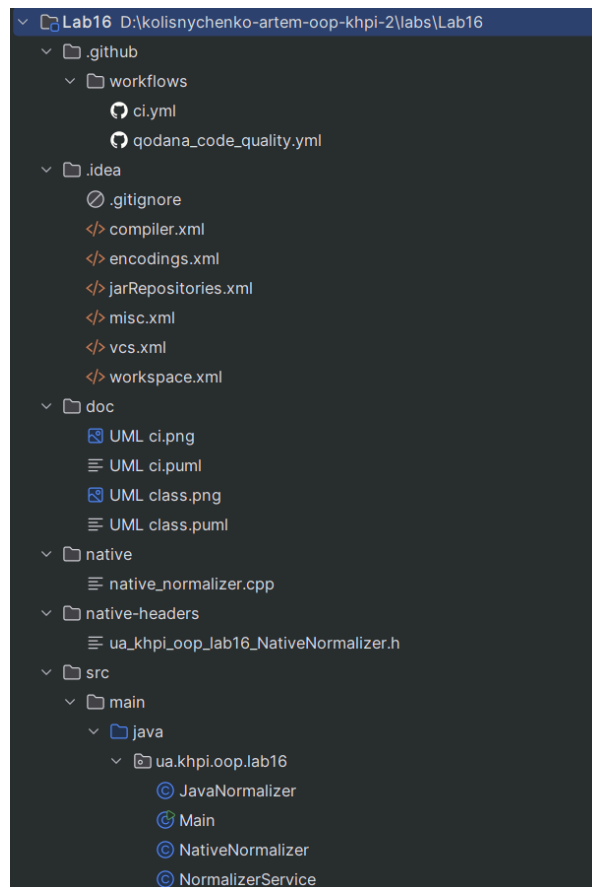


Рисунок 1, 2 — Структура проєкту з Java-, native- та CI-частинами.

5. Java-реалізація операції

Java-реалізація знаходиться у класі `JavaNormalizer`.

Метод `normalize()` приймає масив чисел типу `double[]` і повертає новий масив, у якому всі значення приведені до діапазону від 0 до 1.

Основні кроки Java-реалізації:

1. перевірити, що масив не є `null` і не є порожнім;
2. знайти мінімальне значення;
3. знайти максимальне значення;
4. обчислити нормалізовані значення;
5. повернути результат.

Якщо `min == max`, програма повертає масив нулів.

6. JNI та native-частина

Для зв'язку Java-коду з C++ використано JNI.

Клас `NativeNormalizer` містить native-метод:

```
public native double[] normalize(double[] values);
```

Native-бібліотека завантажується через:

```
System.loadLibrary("nativenormalizer");
```

Header-файл було згенеровано командою:

```
javac -h native-headers -d target\classes  
src\main\java\ua\khp\oop\lab16\NativeNormalizer.java
```

У файлі `native_normalizer.cpp` реалізовано JNI-функцію, яка отримує `jdoubleArray`, знаходить мінімальне та максимальне значення, формує новий масив і повертає його назад у Java.

Для локального запуску на Windows було зібрано бібліотеку:

`nativenormalizer.dll`

Для GitHub Actions на Linux збирається бібліотека:

`libnativenormalizer.so`

7. Команди збирання native-бібліотеки

Для локального запуску на Windows використовувався компілятор `clang++`.

Команда генерації JNI header-файла:

```
$env:JAVA_HOME="C:\Users\bxqqe\jdk\ms-21.0.11"
```

```
& "$env:JAVA_HOME\bin\javac.exe" -h native-headers -d target\classes  
src\main\java\ua\khipi\oop\lab16\NativeNormalizer.java
```

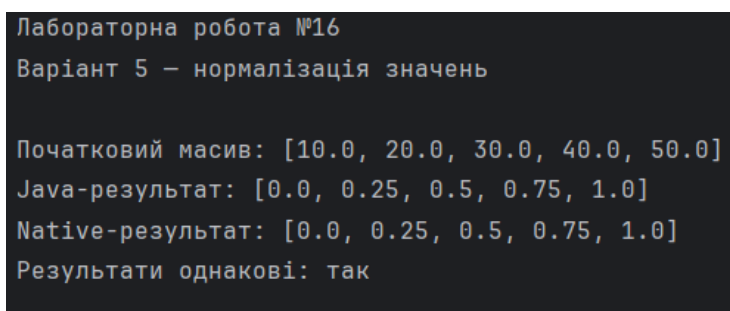
Команда збирання DLL:

```
& "C:\Program Files\LLVM\bin\clang++.exe" -std=c++17 -shared `  
-I"$env:JAVA_HOME\include" `  
-I"$env:JAVA_HOME\include\win32" `  
-Inative-headers `  
native\native_normalizer.cpp `  
-o target\native\nativenormalizer.dll
```

Після виконання команди у каталозі target/native було створено файли nativenormalizer.dll і nativenormalizer.lib.

8. Демонстрація запуску програми

Для перевірки працездатності проєкту було виконано запуск класу Main.



```
Лабораторна робота №16  
Варіант 5 — нормалізація значень  
  
Початковий масив: [10.0, 20.0, 30.0, 40.0, 50.0]  
Java-результат: [0.0, 0.25, 0.5, 0.75, 1.0]  
Native-результат: [0.0, 0.25, 0.5, 0.75, 1.0]  
Результати однакові: так
```

Рисунок 3 — Демонстрація запуску програми Main.

Це підтверджує, що native-бібліотека коректно завантажується, а Java- і C++-реалізації виконують одну й ту саму операцію.

9. Тестування

У проєкті створено тестовий клас `NormalizerTest`.

У тестах перевіряється:

- коректність Java-реалізації;
- коректність native-реалізації;
- однаковість результатів Java та native-реалізацій;
- граничний випадок, коли всі значення однакові;
- некоректні вхідні дані для Java-реалізації.

```
[INFO] Running ua.khpi.oop.lab16.NormalizerTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.047 s -- in ua.khpi.oop.lab16.NormalizerTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
```

Рисунок 4 — Результат виконання модульних тестів.

10. UML-діаграми

Для проєкту було створено UML-діаграму класів і діаграму CI-процесу.

Діаграма класів показує зв'язок між класами `Main`, `JavaNormalizer`, `NativeNormalizer` і `NormalizerService`, а також native-функцією у C++.

Діаграма CI показує локальний і віддалений цикл перевірки: редагування коду, генерацію JNI header-файла, збирання native-бібліотеки, запуск тестів і виконання workflow на GitHub Actions.

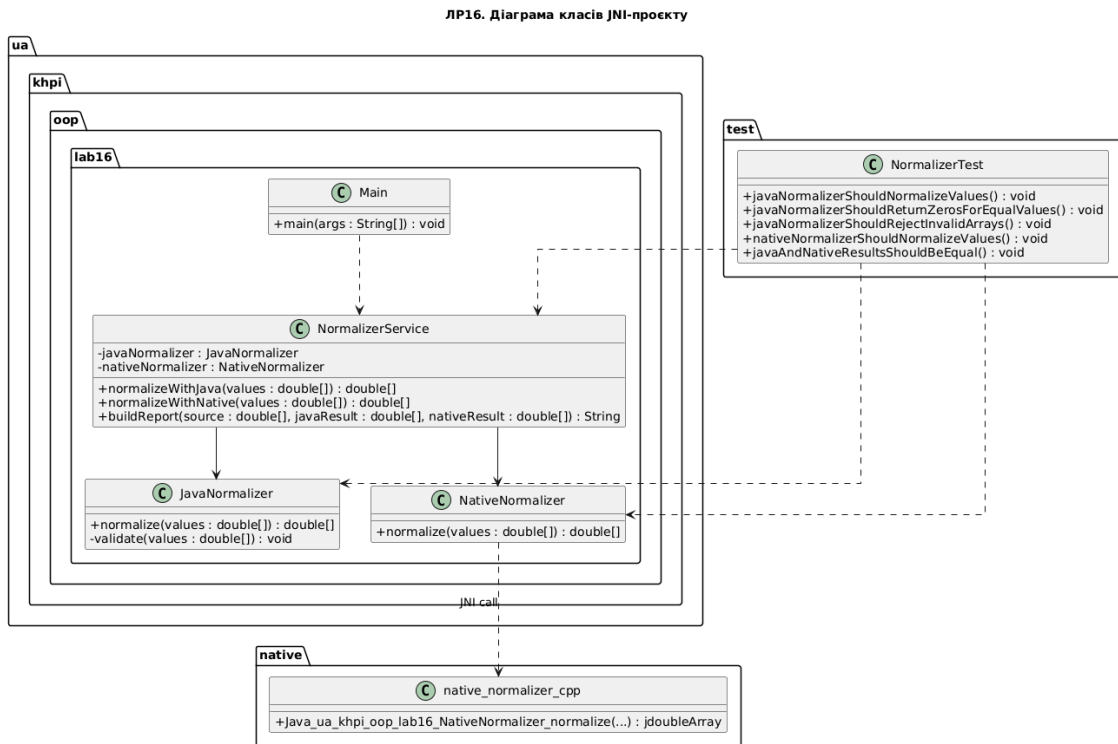


Рисунок 5 — UML-діаграма класів JNI-проєкту.

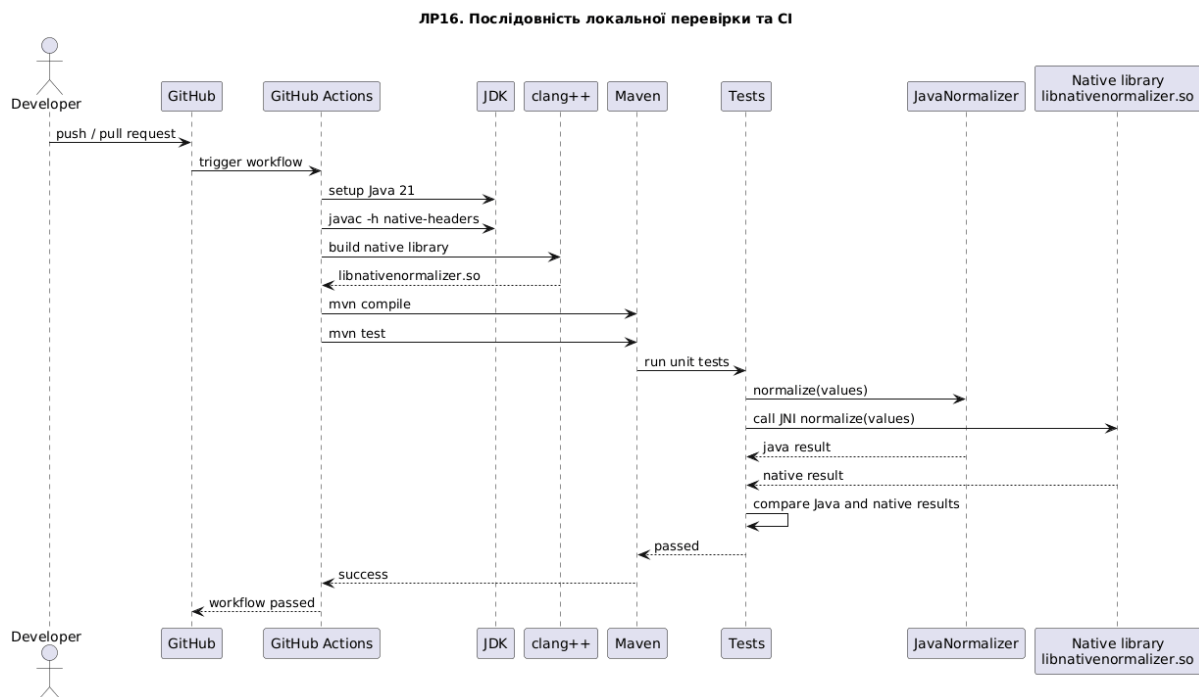


Рисунок 6 — UML-діаграма локальної збірки та CI через GitHub Actions.

11. Налаштування workflow GitHub Actions

Для автоматизованої перевірки проєкту було підготовлено workflow GitHub Actions.

Workflow знаходиться у файлі:

`.github/workflows/ci.yml`

Він запускається після `push`, `pull_request` та вручну через `workflow_dispatch`.

У процесі виконання workflow відбуваються такі кроки:

1. отримання коду з репозиторію;
2. налаштування JDK 21;
3. генерація JNI header-файла;
4. збирання native-бібліотеки через `clang++`;
5. збирання Java-проєкту;
6. запуск модульних тестів.

Успішне виконання workflow підтверджує, що проєкт коректно перевіряється не лише локально, а й у відтворюваному віддаленому середовищі.



4 workflow runs		Workflow ▾	Event ▾	Status ▾	Branch ▾	Actor ▾
✓	Lab16 Java JNI CI #4: Pull request #18 synchronize by QewBye48	lab16	23 minutes ago	38s	...	
✓	rename lab16 maven file to pom Java JNI CI #3: Commit 278cce3 pushed by QewBye48	lab16	23 minutes ago	24s	...	
✗	Lab16 Java JNI CI #2: Pull request #18 synchronize by QewBye48	lab16	27 minutes ago	19s	...	
✗	add lab16 ci workflow Java JNI CI #1: Commit ee85e9a pushed by QewBye48	lab16	27 minutes ago	22s	...	

Рисунок 7 — Успішне виконання workflow GitHub Actions.

12. Технічна проблема та її усунення

Під час виконання лабораторної роботи виникло кілька технічних проблем, пов'язаних із JNI та Maven.

Перша проблема — помилка `UnsatisfiedLinkError`. Вона виникала через те, що JVM не знаходила native-бібліотеку `nativeNormalizer`. Для усунення проблеми було зібрано файл `nativeNormalizer.dll` у каталог `target/native` і вказано шлях до нього через параметр:

`-Djava.library.path=target/native`

Також у класі `NativeNormalizer` було налаштовано завантаження бібліотеки з каталогу `target/native`, щоб тести Maven могли знайти DLL під час запуску.

Друга проблема виникла у GitHub Actions. Workflow падав на кроці `mvn compile`, тому що Maven не знаходив POM-файл. Причина полягала в тому, що файл Maven мав назву `Lab16.xml` замість стандартної назви `pom.xml`.

Для виправлення файл було перейменовано на `pom.xml`. Після цього workflow успішно виконав генерацію JNI header-файла, збирання native-бібліотеки, збирання Java-проєкту і запуск тестів.

13. Висновок

У лабораторній роботі було створено Java-проєкт із JNI та автоматизовано перевірку через GitHub Actions.

Операцію нормалізації масиву до діапазону 0..1 реалізовано у двох версіях: звичайною мовою Java та мовою C++ через JNI.

Для native-частини було згенеровано JNI header-файл, реалізовано C++ функцію і зібрано native-бібліотеку. Java-програма успішно завантажує цю бібліотеку та викликає native-метод.

Модульні тести підтвердили коректність Java-реалізації, native-реалізації та однаковість їхніх результатів для однакових вхідних даних.

GitHub Actions workflow успішно виконав віддалену перевірку проєкту: налаштував JDK, згенерував JNI header, зібрав native-бібліотеку, скомпілював Java-проєкт і запустив тести.

Отже, лабораторна робота демонструє практичне застосування Maven, JNI, native C++ коду, модульного тестування та безперервної інтеграції для Java-проєкту з native-залежністю.